17-214/514: Agentic Software Development

Test Design and Coverage

Hammad Ahmad and Claire Le Goues • Wednesday, September 2, 2026 • Lecture

Administrivia

- Please fill out the intake form (linked on Canvas homepage) to get set up for this course.
  - You cannot get started with A1 without us creating your repos for you.
  - If you have a pending A1 invite, accept it before the link expires.
- For labs, make sure you are done with the lab *before* you go in for recitation (barring any roadblocks).
- If you enrolled in the course recently and missed previous "quizzes", email me and I'll mark you as excused. For Lab 1, you can show your solution to your TA during recitation.

What did we talk about last time?

- Testability is a design property. Controllability and observability, decided before any test is written.
- Inject the dependency, split the rule from the I/O, return something you can check, etc.
- Generated tests arrive in bulk. Read them like a skeptic.
  - Generated suites are weakest at the edges.
- Today: **which** tests to write, and what the coverage number does and does not tell you.

Learning goals for today

- Write a specification (preconditions, postconditions, invariants) precise enough to generate from **and** verify against
- Design test cases using equivalence classes and boundary values
- Use property-based testing to check invariants across generated inputs
- Read coverage metrics critically, and explain what they do **not** tell you
- Use mutation testing to ask whether your tests would actually catch a bug

All 312 tests pass

booking-service — npm test

```
$ npm test

 RUN  v2.1.9 ~/projects/booking-service

 ✓ test/room-repository.test.ts (40 tests) 3ms
 ✓ test/waitlist.test.ts (40 tests) 4ms
 ✓ test/time-interval.test.ts (40 tests) 3ms
 ✓ test/api.test.ts (40 tests) 7ms
 ✓ test/clock.test.ts (40 tests) 4ms
 ✓ test/booking-service.test.ts (40 tests) 6ms
 ✓ test/booking-rules.test.ts (40 tests) 4ms
 ✓ test/notifier.test.ts (32 tests) 4ms

 Test Files  8 passed (8)
      Tests  312 passed (312)
   Start at  16:09:17
   Duration  262ms
```

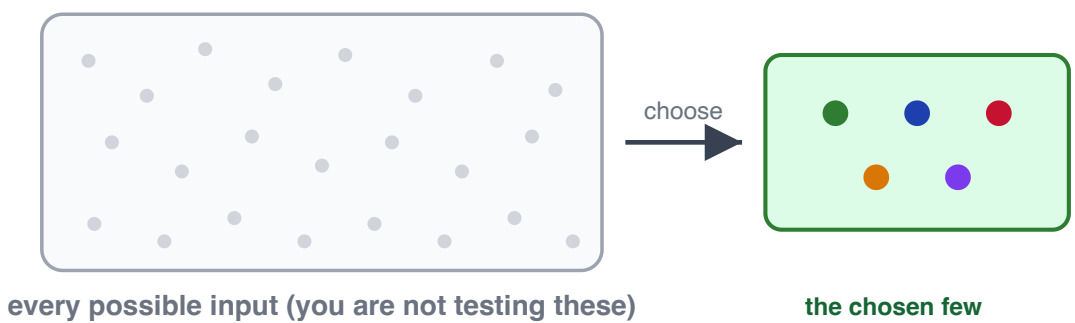
But, is the software correct?

- Correct according to **what**?
- 312 tests could be 312 happy paths, or 312 tests that copied the answer from the code. The count alone tells you nothing.
- Two questions:
  - Are these the **right** claims?
  - Would these tests **notice** a bug?

You can’t test everything

Infinite inputs, finite semester

- `add(a: int, b: int)` has roughly four billion times four billion possible inputs (assuming 32-bit `int`s).
- How about `send_email(address: str, body: str)`?
- Can you test all of them?
- Testing effectively means choosing a **few** inputs that stand in for the rest.



Specifications

What's in a specification?

one operation, one contract

preconditions *the caller's half*
what must be true BEFORE the call

postconditions *the implementation's half*
what the operation promises AFTER

invariants *everyone's problem*
hold ALWAYS, before and after every operation

together they form a contract, and every clause of it can be checked

Consider a bank account

```
withdraw(amount)
  requires:  amount > 0
             amount <= balance
  ensures:   balance' = balance - amount
  invariant: balance >= 0
             (always, after every operation)
```

- Every line is a claim a test can check.
- It says nothing about **how**. No data structures, no loops.

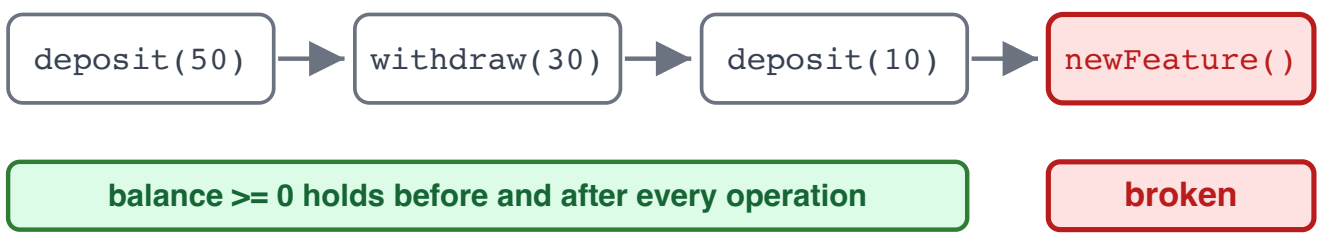
Who checks the preconditions?

`withdraw` requires `amount > 0`. Someone calls it with `-50`. Whose bug?

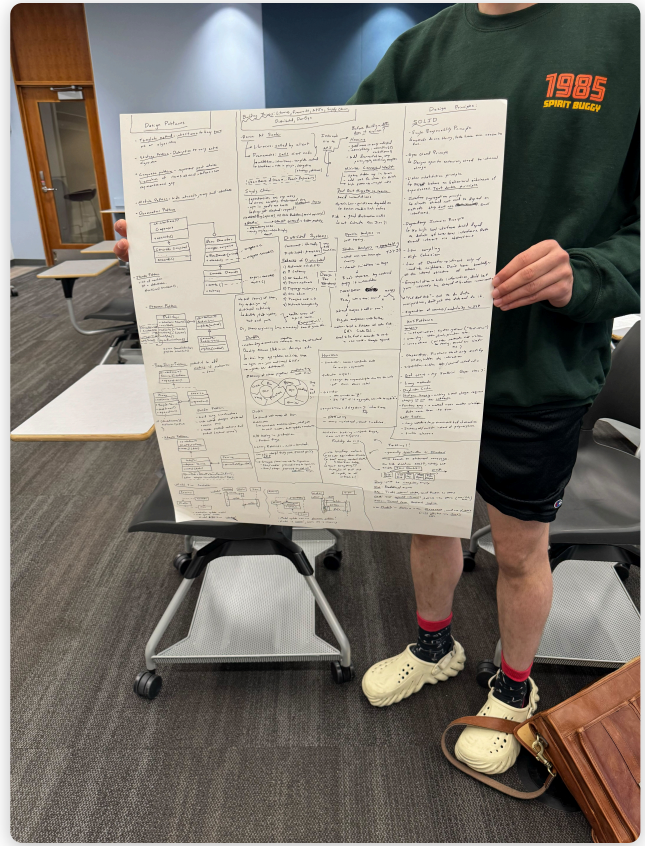
- The caller's. They broke the contract.
- Should `withdraw` check anyway?
- At a public boundary (users, other teams, the network), yes. **Validate and fail loudly**.
- Deep inside your own module? **It depends**.
  - Rule of thumb: validate where trust ends.

Invariants

- A postcondition binds one operation. An invariant binds **all of them**.
- "Balance never negative." "The cache never exceeds capacity." "Every ID is unique."
- One operation that forgets it breaks the invariant.
- FWIW: Humans and agents are both great at forgetting invariants.
  - (For an agent, the rule only has to scroll out of its context.)



Be precise about your spec



"Sorts the list" is not *really* a spec

Vague

- sorts the list
- handles errors gracefully
- should be fast

Precise

- returns a new list, ascending, stable; input unchanged; empty allowed; throws on null
- returns [] when no matches; throws on null query
- responds in < 200ms for inputs up to 10,000 items

Could each clause become a test? "Gracefully" cannot. `[]` on no matches can.

Whose job is the spec?

- An agent can **satisfy** a spec. Choosing one takes judgment about users, risk, and trade-offs.
- Leave the spec vague and the agent fills the gaps with guesses. (Confident ones!)
- "Make it work" hands every decision to the agent. You probably (read: **definitely**) don't want to do that.
  - Adding "make no mistakes" does not help. :-)

A spec often has two jobs

Audit the suite against the spec

- Walk the contract clause by clause. Does a test violate each precondition, assert each postcondition, probe each invariant?
- The clauses without a test are your gap list.
- This is **specification coverage**. Did every claim get a test? (Don't worry about lines of code yet.)
- The walk is mechanical, so you can let the agent do it.
  - The spec, the verdict on each match, and the gaps you choose to leave are your decision.

Choosing cases from the spec

Equivalence classes

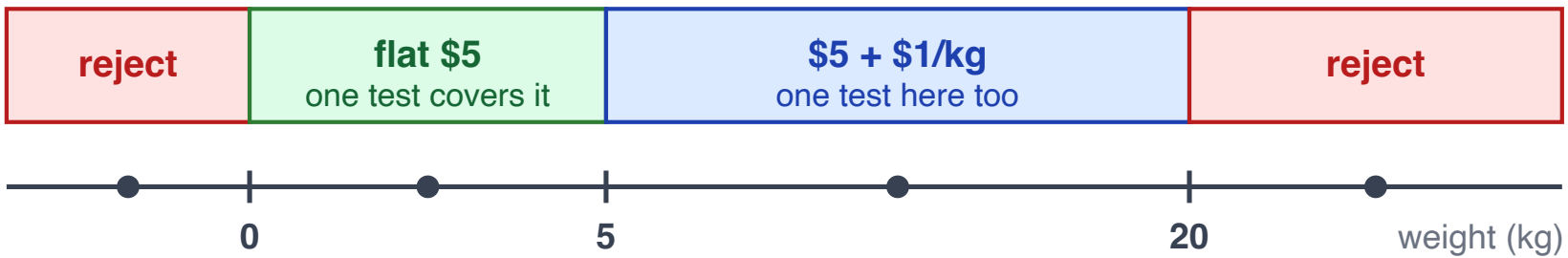
- Partition the inputs into groups that **behave the same**.
- Test one representative from each group.
- E.g., for a shipment (next slide), if 0-5 kg all ship at the same rate, one test covers them.

Example: shipping cost

```
function shippingCost(weightKg: number): number {
  if (weightKg <= 0) throw new Error("invalid weight");
  if (weightKg <= 5) return 5;                    // flat rate
  if (weightKg <= 20) return 5 + (weightKg - 5); // $1 per kg over 5
  throw new Error("too heavy");
}
```

Which inputs would you test? How many?

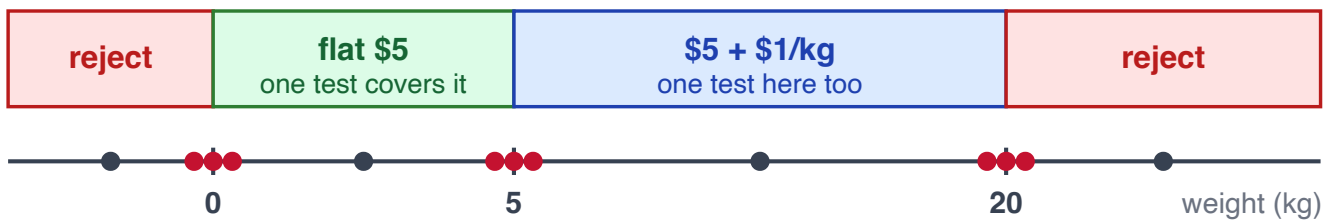
The four classes



Four tests for the four equivalence classes.

Boundary-value testing

- Bugs live at the edges. `<` vs `<=`, off-by-one, empty, null, the maximum value.
- The interesting inputs are 0, 5, and 20, and one step on either side of each.
- Per class, three probes. A normal value, the boundary, and one step past it.

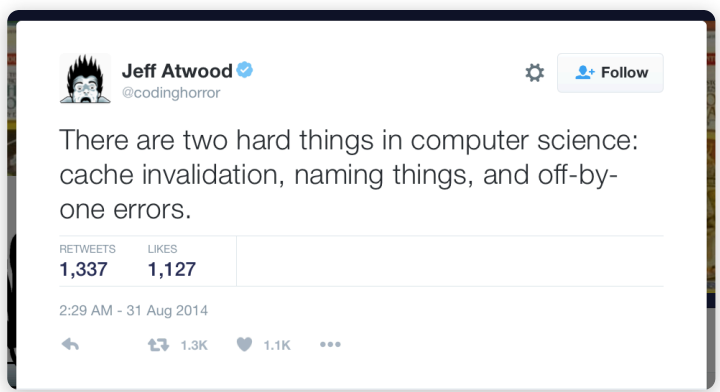


the red probes are new: on each edge, and one step to either side

The classic off-by-one

```python
def can_vote(age):
    return age > 18   # should this be >= ?
```

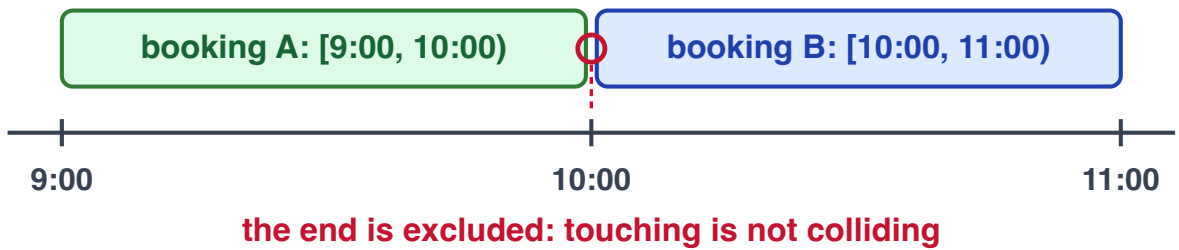
```python
def test_minor():  assert can_vote(17) is False   # passes
def test_adult():  assert can_vote(19) is True    # passes
```



https://halfelf.org/2016/the-problem-with-renaming/

Interval boundaries

Bookings run over `[start, end)`. Do 9:00-10:00 and 10:00-11:00 overlap?



- Three tests: touching, one minute inside, one minute apart.
- Monday's activity suite didn't build the touching case. Consequence: `<=` shipped instead of `<`.
- Boundary tests can enforce the convention. Without them, callers can invent their own.

Classes plus boundaries

```
shippingCost(weightKg)
  <= 0      invalid
  (0, 5]    flat $5
  (5, 20]   $5 + $1 per kg over 5
  > 20      invalid
```

- Classes give you the four regions.
- Boundaries give you the dangerous inputs. 0, 5, 20, and one step either side.
- A dozen tests cover the space and probe every edge.

Several parameters. How do you combine them?

Back to the bank account:

```
function pay(cost: number, useCredit: boolean): boolean {
  if (useCredit && credit >= cost) { credit -= cost; return true; }
  if (cash >= cost)                { cash   -= cost; return true; }
  return false;
}
```

- Three conditions decide everything: `useCredit`, `credit >= cost`, `cash >= cost`.
- Each one is only ever true or false. A test just has to set it up either way.
- The input space collapses to three booleans. Which combinations do you test?

Which combinations?

```
function pay(cost: number, useCredit: boolean): boolean {
  if (useCredit && credit >= cost) { credit -= cost; return true; }
  if (cash >= cost)                { cash   -= cost; return true; }
  return false;
}
```

useCredit	enoughCredit	enoughCash	result
T	T	-	paid (credit)
T	F	T	paid (cash)
T	F	F	false
F	-	T	paid (cash)
F	-	F	false

- Drop impossible rows, dash what doesn't matter.
- Two invalid inputs at once rarely need their own test.

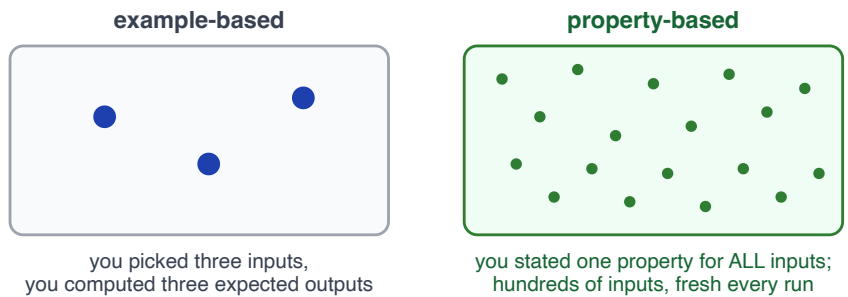
How many boundary cases is enough?

It depends.

Property-based testing

State a property, generate the inputs

- Example-based: **you** pick the input and the expected output. One claim per test.
- Property-based: you state what must hold for **any** input, and the framework generates hundreds of inputs trying to break it.
- e.g., `reverse(reverse(xs)) == xs`, for any list. The generator finds the weird input for you.



A property for our bank account

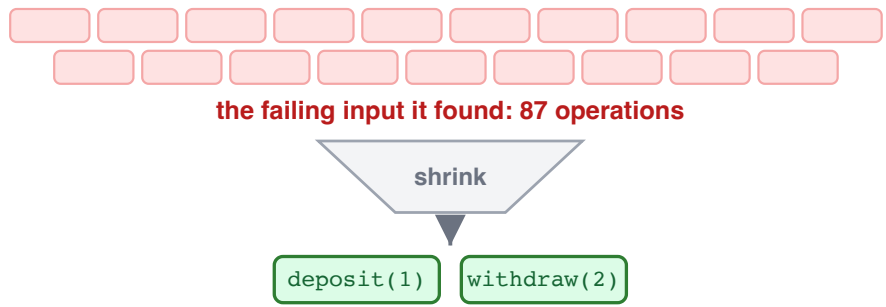
The invariant from our spec:

```
@given(ops=lists(operations()))
def test_balance_never_goes_negative(ops):
    acct = BankAccount(0)
    for op in ops:              # random deposits and withdrawals
        acct.try_apply(op)
    assert acct.balance >= 0    # the invariant, after ANY sequence
```

One property can have hundreds of generated sequences per run.

When it fails: shrinking

- The first failure might be an 87-operation sequence. You don't get that one.
- The framework **shrinks** it to the smallest input that still fails.
- You get `[deposit(1), withdraw(2)]` as the bug report.



Where properties shine

- **Round trips:**
  - `decode(encode(x)) == x`
  - `parse(print(x)) == x`
- **Algebraic laws:** adding then removing an item restores the collection.
- **Invariants under sequences:** the account balance, the cache bound, the uniqueness rule.
- Awkward fit: "the output should look right." No property, no property test.

Write the tests first, then generate code

Recall test-driven development from Monday. Here is the cycle, with an agent in it:

Red: you + agent
A test that fails because the code does not exist yet. That is the definition of done.

Green: the agent
Make it pass. The target was fixed before it started.

Refactor: agent + you
Clean up with the test standing guard. The step everyone skips once green is cheap.

- A property makes an especially good red. Fresh inputs every run, nothing to memorize.
- The agent can **propose** tests. Which ones are the contract is still your call.

Test what it claims, or what it does?

Specification-based (black-box)

- Cases come from the contract.
- The code is a black box. Rewrite it and the tests still apply.
- Blind spot: what the spec forgot to say.

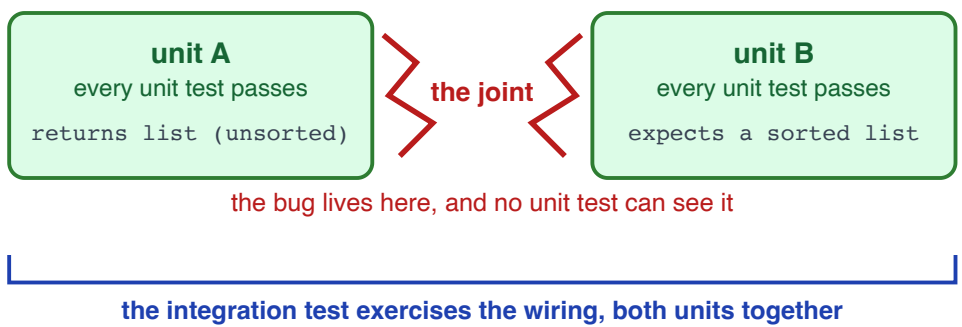
Structural (white-box)

- Cases come from the code.
- Finds the corner the spec never mentioned.
- Blind spot: the code is its own oracle.

You want both!

All units pass. The system is still broken. How?

- Each piece works alone. But the system falls apart when pieces interact.
- The bug lives in the **joints** (interaction points). Between modules, at the database, across the API.
- Unit tests can't see joints. Integration tests can. Slower, fewer, and they often blame less precisely.



Code coverage

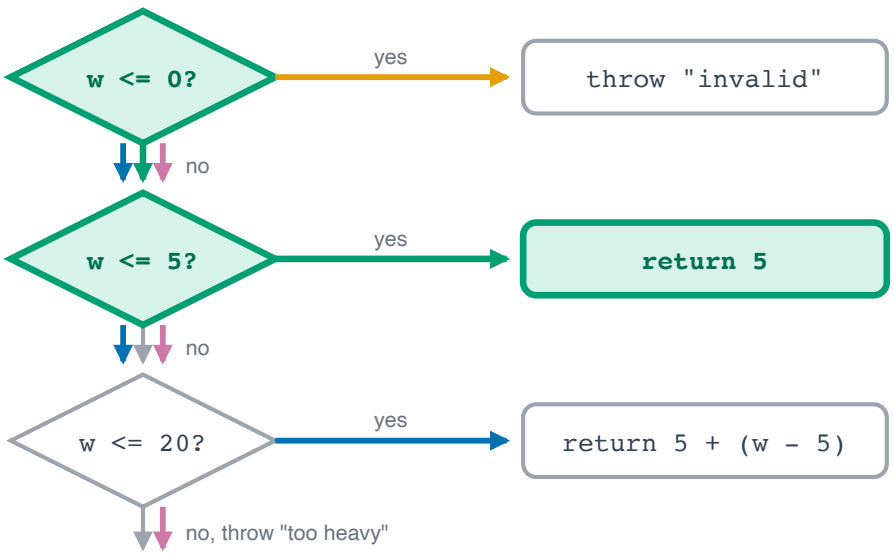
Coverage: what did your tests actually run?

Let's trace: `expect(shippingCost(3)).toBe(5)`.

```
function shippingCost(weightKg: number): number {
  if (weightKg <= 0) throw new Error("invalid weight");
  if (weightKg <= 5) return 5;
  if (weightKg <= 20) return 5 + (weightKg - 5);
  throw new Error("too heavy");
}
```

Statement coverage: which lines ran. 3 of 6.

Branch and path coverage



The "walk" for `shippingCost(3)`:

Branch coverage counts arms. Did each decision go both ways? 2 of 6.

Path coverage counts whole walks. Four possible, we took the green one. 1 of 4.

(**Loops** add a back-edge, so paths may never end. Test zero, once, many.)

Statement or branch?

```
String label(int n) {
  String s = "neutral";
  if (n > 0) s = "positive";
  return s;              // what about n <= 0?
}

@Test void t() { assertEquals("positive", label(5)); }
```

- Every line executes. 100% statement coverage.
- But the `if` only ever went true. The `n <= 0` branch is untested, so 50% branch coverage.
- The bug lies in the branch we never took.

How useful is a coverage report?

Consider the following JaCoCo reports:

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Palindrome.java		100%		100%	0	5	0	7	0	2	0	1
Total	0 of 38	100%	0 of 6	100%	0	5	0	7	0	2	0	1

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Palindrome		21%		17%	3	5	4	7	0	2	0	1
Total	30 of 38	21%	5 of 6	17%	3	5	4	7	0	2	0	1

https://www.baeldung.com/jacoco

- Red is the useful part. Nobody tested those 30 instructions.
- Green only says the code ran. Not that the runs were useful.

What coverage misses

100% coverage is not 100% correct

```python
def absolute(x):
    return x          # bug: never negates

def test_absolute():
    absolute(-5)      # executes the line: 100% coverage!
    assert True       # ...asserts nothing
```

Every line ran. Zero behavior was checked.

Can 100% branch coverage miss a bug?

```python
def scale(a, z):
    x = 5
    if a <= 1:
        x = a - 1
    return z / x

def test_small(): assert scale(0, 10) == -10   # takes the if
def test_large(): assert scale(5, 10) == 2     # skips it
```

- 100% branch coverage. Both tests pass.
- Try calling `scale(1, 10)`.

Goodhart's Law

> "When a measure becomes a target, it ceases to be a good measure."
>
> - Charles Goodhart

Consequence: make 100% coverage the goal and you get tests that hit lines rather than catch bugs.

"Get me to 100% coverage"

- Your agent can very likely achieve this. Often as a pile of tests that execute a bunch of stuff and check *some stuff* (but not everything).
- The number turns green. Most of the bugs stay where they were.
- Coverage measures **execution**, not checking.

So what coverage number should you chase?

Coverage gates

Many teams reject a PR if coverage drops below a threshold.

100% can be impossible

Unreachable branches, generated code. Accept the gap and say why.

- Treat coverage as a floor.
  - Low coverage is a warning. High coverage is not job done.
- Aim for the uncovered **critical paths**, not (e.g.,) the last few getters and `toString()`.
- (Generally, sharp assertions at 80% are better than boilerplate assertions at 100%.)

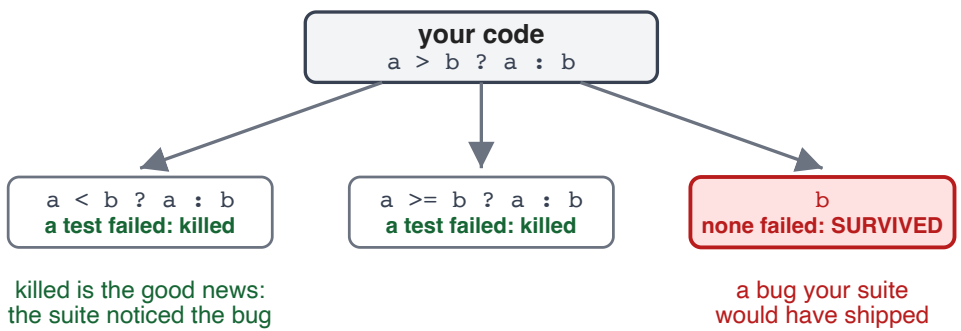
Mutation testing

Would your tests notice?

- Coverage asks: "did your tests **run** this code?"
- A different question: "would this test fail if the code were wrong?"
- Mutation testing can help here!

How it works

- Inject small bugs, called mutants. Change `+` to `-`, `<` to `<=`, `true` to `false`.
- Re-run the suite. A good suite **kills** the mutant (i.e., some test fails).
- A mutant that **survives** is a bug your tests would have shipped to prod.



inject one small bug at a time, re-run the suite

A mutant your tests should catch

```
const max = (a: number, b: number) => a > b ? a : b;
// mutant:                            a < b ? a : b;
```

If no test fails when `>` flips to `<`, your `max` tests are not doing their job.

Mutation score

- The percentage of mutants your suite kills.
- A harder number than coverage, but (arguably) a more useful one.
- Mutation testing is expensive, though.
  - It re-runs your tests for every mutant, so spend it where correctness matters most.

More tests = more better?

A test-design recipe

- **Start from the spec:** every clause is a claim, and every claim deserves a test.
- **Partition** the inputs into equivalence classes. Invalid inputs are classes too.
- **Probe the boundaries** of every partition, on and around the edge.
- **State properties** for the invariants and round trips, and let the generator hunt.
- **Interrogate the result:** would this suite notice a bug? (The mutation-testing question, with or without the tool.)

> **"How much of this can my agent do?"**

Every step, as a draft. The spec, the call on which classes matter, and the verdict on whether it is enough stay with you.

What you deliberately do NOT test

- Infinite input space, finite semester. **Not** testing something is a decision, so make it on purpose and write it down.
- A reasonable claim: "Not testing concurrent access. The service is single-threaded by design."
  - (That is a rationale.)
- A not-so-reasonable claim: "We didn't get to it" or "Claude said it was enough".
  - (At best, that is a confession.)

Reminder: every assignment grades your verification rationale (what's covered, what deliberately isn't, and why).

Let's practice: design the suite

```
reserveUsername(name)
  requires: name is 3–15 chars, lowercase letters and digits only
  ensures:  name is registered to the caller;
            any later reserveUsername(name) throws AlreadyTaken
```

Work with your neighbors to sketch the suite:

- The equivalence classes (don't forget the invalid ones).
- The boundary cases you'd insist on.
- One **property** not shown on the slide that is worth generating inputs for. (Maybe the spec is silent about something?)

Canvas → Quizzes → Lecture 4 In-Class Quiz
Code: "ilovetesting"

So how much testing is enough?

It depends.

Summary

- The claims come from the spec. Deciding what it says is your job. The agent builds from it and the suite checks against it.
- Choose cases on purpose. Equivalence classes, boundaries, and properties for what you can't enumerate.
- Coverage tells you what you **didn't** run, and nothing about correctness.
- Mutation testing asks whether these tests would catch a real bug.
- Quality over quantity, and write down what you deliberately don't test.

What's next

W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14
A1 out		A1 due		A2 due · M1		A3 due		A4 due	M2		A5 due		A6 due

- **Friday, Lab 2:** write the property that catches what a green, high-coverage suite missed, then audit that suite.
  - Reminder: we expect you to have completed the lab **before** going in for recitation.
- **Monday:** no class (Labor Day), but Assignment 1 is due at 11:59pm.
  - Your A1 suite will be mostly top-of-pyramid (README claims, over HTTP). The testability fix lets some of the testing move down the pyramid.
- **Wednesday:** what is a software system's design?